Focused AI-Assisted Engineering Assessment: Course Waitlist

Objective

Complete one bounded course-registration feature with AI used as an engineering assistant. Demonstrate clear task decomposition, thoughtful review of AI output, and independent verification. This exercise is about a reliable service slice, not a complete registration platform.

Problem Statement

Implement enrollment, dropping, and FIFO waitlist promotion for one course offering in the provided starter project. A learner may enroll only once. When the course is full, eligible learners join the waitlist. If a seat opens, promote the earliest eligible learner. Use the starter's data model; if none is provided, a small service or command-line prototype with an in-memory store is sufficient.

State the prerequisite and waitlist assumptions. In particular, define whether an ineligible learner is skipped or remains waiting, and apply that policy consistently.

Acceptance Criteria

1. Enrollment succeeds when capacity remains and the learner satisfies the stated prerequisite rule.
2. When full, an eligible learner joins the waitlist once; duplicate enrollment requests do not create duplicate records.
3. Dropping an enrolled learner opens a seat and promotes the earliest eligible waitlisted learner according to the documented eligibility policy.
4. An ineligible learner is not enrolled or promoted. The documented policy for skipping or retaining that learner is applied consistently.
5. Invalid requests and duplicate drops return clear results without corrupting enrollment, capacity, or waitlist state.
6. Tests cover available capacity, full capacity, duplicate requests, FIFO promotion, prerequisite failure, and invalid or repeated drops. If the starter uses concurrent requests, preserve its transaction or locking pattern and add a focused concurrency test where practical.
7. Use AI for at least one engineering activity. Record the prompts used, relevant AI response or summary, whether each suggestion was accepted, edited, or rejected, and how the resulting work was verified.

Scope Boundaries

- Implement only one offering's enrollment, drop, and waitlist-promotion slice; do not build the full registration product.

- Do not add a frontend, course catalog, payments, identity integrations, multi-term registration, deployment, or a new database unless the starter project requires it.
- Do not implement separate greenfield, brownfield, and ambiguous scenarios. Work in the supplied starter; otherwise use the minimal service or CLI fallback described above.
- State the prerequisite and waitlist assumptions; do not build a configurable policy engine.

Deliverables

- Runnable code for the scoped feature and focused automated tests.
- A design specification describing the service boundary, relevant data/API design, enrollment and capacity invariants, waitlist-promotion rule, concurrency or persistence approach, and key assumptions or tradeoffs. Keep it proportionate to this bounded feature.
- An AI prompt and use record containing the prompts used, relevant response excerpts or summaries, how suggestions were accepted, edited, or rejected, and the files or decisions they informed. Include verification evidence for AI-assisted changes.
- Brief setup and run instructions, including the commands used to execute the tests.
- A concise summary of limitations or follow-up risks discovered within scope.

Candidate Expectations

- Start by understanding the supplied project and its existing interfaces, persistence, and test conventions. If no starter is supplied, use the minimal service or CLI fallback; do not build a frontend.
- State the prerequisite and waitlist assumptions, including what happens when an ineligible learner reaches the front of the queue.
- Use AI for at least one engineering activity, but independently review and verify any AI-generated suggestion before relying on it. Be candid about what AI contributed and what you changed or rejected.
- Do not include credentials, secrets, or unnecessary sensitive data in prompts or submitted prompt records.
- Prioritize correct capacity and queue behavior, focused tests, understandable changes, and evidence of independent review over feature count or extra infrastructure.
- Submit the implementation and all listed deliverables together so the work can be run and reviewed.

Suggested Timebox

Suggested effort: 7-8 hours. Focus the time on the scoped workflow, its tests, the design specification, and a clear record of AI use and verification. Use the AI-Assisted scorecard in Course-Registration-Rubric-Score.md, rating evidence relevant to this task's scope.